

Optimisation de l'exécutions des Plans de requêtes en SQL

Eliott Paquet, MPI

17 mai 2026

Table des matières

1	Introduction	2
1.1	Abstract	2
1.2	Fonctionnement du traitement d'une requête SQL par un SGBD . .	2
1.3	L'Algèbre relationnelle	3
2	Optimisation statique	4
2.1	Descente de Sélection	4
2.2	Descente de Projection	5
3	Optimisation dépendante des données	6
3.1	Optimisation des Sélections	6
3.2	Choix de l'algorithme de jointures	8
3.3	Optimisation de l'ordre d'exécution des jointures	9
4	Nos Résultats	10
4.1	Présentation du jeu de donnée	10
4.2	Analyse des résultats	10
5	Conclusion	10
5.1	Conclusion	10
5.2	Amélioration du modèle actuel	10
5.3	Autre Champ d'étude pour l'optimisation de notre SGBD	10
6	Annexe	10
6.1	Présentation Sommaire du code	10
6.2	Gestion globale	10
6.3	Gestion mémoire des tables	10

6.4	Gestion du plan	10
6.5	Descriptions rapides du code utilisé	10

1 Introduction

1.1 Abstract

Depuis les dernières décennies la gestion des données est devenue un élément essentiel. Pour se faire les ingénieur.e.s et chercheur.e.s dans ce domaine ont développé de nombreux outils, dont le SQL qui permet de formuler les requêtes à envoyer à un SGBD, les Systèmes de Gestion de Base de Donnée pour interpréter le SQL et stocker ces données et le modèle relationnel qui permet de définir différente relation entre nos données et qui est étudié ici. C'est dans ce contexte qu'à travers ce TIPE, nous analysons certaine manière utilisée par les SGBD pour optimiser la vitesse d'exécution des requêtes SQL. Nous allons étudier dans un premier temps, à travers l'algèbre relationnelle des optimisations de plan et certaine amélioration algorithmique des techniques de jointures et ensuite nous analyserons les résultats de ces optimisations sur notre SGBD, prénommé **BDD-TIPE**, créé pour ce projet, en C++, disponible publiquement sur Git-Hub, permettant de comparer l'efficacité de ces optimisations.

Ce TIPE a été conçu en binôme. Mon binôme, Antoine Bertin-Paillette, s'est concentré sur la gestion mémoire de nos données avec le même objectif et sur l'implémentation d'un parser et de la gestion mémoire dans notre SGBD. La particularité de son analyse repose sur le paradigme orienté colonne du stockage des données qui fut initialement le cœur de notre étude. Ce paradigme n'as eu au final que des impacts mineurs sur mon sujet d'études décrit dans 6.3.

1.2 Fonctionnement du traitement d'une requête SQL par un SGBD

La requête SQL qu'un client envoie au SGBD est d'abord transformé par le parser en un arbre d'information. L'Algebrizer transforme cette liste de données en un arbre primaire de nœud composé de Sélections, Projections ou jointures. Cet arbre est ensuite optimisé en fonction des décisions du SGBD pour être enfin exécuté, c'est à ce moment-là que les données sont récupérer, transformer en envoyer au client.

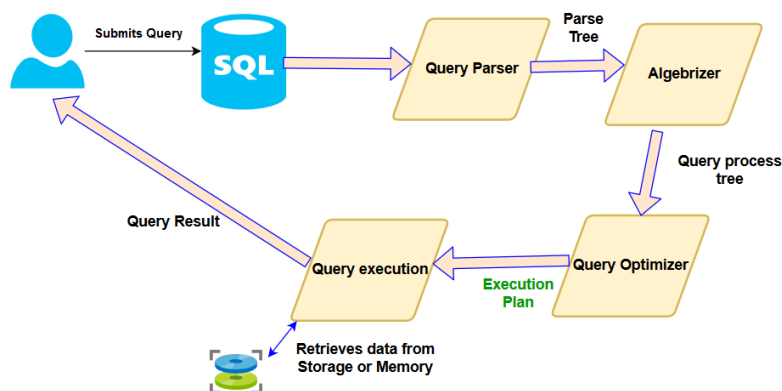


FIGURE 1 – Schéma explicatif du traitement d'une requête SQL

1.3 L'Algèbre relationnelle

En 1971, Edgar F. Codd a démontré que toute requête du calcul relationnel, c'est-à-dire la logique du premier ordre sans symbole de fonction peut s'écrire de manière équivalente comme requête d'algèbre relationnelle. Le SQL permet de décrire une requête de l'algèbre relationnelle en plus de certains ajouts avec par exemple : **MIN**, **MAX**, **COUNT**, **AVG**.

L'enjeu de l'Algebrizer est donc de convertir cette requête SQL en un arbre d'opérateur de l'algèbre relationnelle pour ensuite utiliser les outils déjà existants d'optimisations ces arbres. Les trois principales opérations que possède l'algèbre relationnelle sont :

- Les Selection (σ) : Les selections représente une condition appliqué sur une colonne.exemple : $\sigma_{Dp.name='Morbihan'}$
- Les projection (Π) : Les projections permet de faire disparaître des colonnes à un certain moment de l'exécution, sont enlevé celle qui ne sont pas mentionnée. Exemple : $\Pi_{Sales.id, Sales.price}$
- Les projection ($\bowtie$) permettent de faire la joiture entre deux table avec comme condition une égalité entre les valeur d'une colonne par table jointe. Exemple : $\bowtie_{Dp.id=Sales.DpId}$

Les Arbres se lisent du bas vers le haut dans le sens des flèches comme avec premier exemple.

```

1 SELECT S.id, S.price
2 FROM Sales
3 JOIN Dp ON Dp.id = S.DpId
4 WHERE Dp.name = 'Morbihan';

```

FIGURE 2 – Exemple de requête classique en SQL

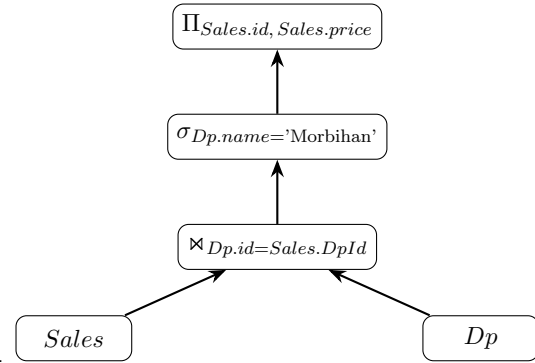


FIGURE 3 – Exemple de conversion par un algebrizer de la requête en 1.3 en un arbre d'opération de l'algèbre relationnelle

On remarquera que l'arbre est binaire, et que seule les opérations de Jointure possède deux enfants. Le premier arbre conçu par un algebrizer qui se veut être la traduction la plus direct de la requête vas être de la forme Projection, Sélection puis Jointures, faisant ainsi toute les jointure d'abord avant d'appliquer la Condition exprimer après le where pour enfin appliquer la projection finale ne renvoyant que les données demandé. Ce premier arbre est techniquement valide ainsi toute transformation équivalente à cette arbre auras comme exécution le résultat attendu.

2 Optimisation statique

Maintenant que les bases de la représentation théorique des requêtes SQL ont été posés, on peut étudier les propriétés que possèdent ces arbres.

2.1 Descente de Sélection

La première analyse que l'on peut remarquer est que la selection est distributive sur la jointure, c'est à dire que sur un table A et B on as :

$$\sigma_{A.a < 100 \text{ and } B.b > 10} (\Join_{A.c = B.d} (A, B)) \iff \Join_{A.c = B.d} (\sigma_{A.a < 100} (A), \sigma_{B.b > 10} (B))$$

L'intérêt de cette remarque réside dans le fait qu'une Jointure n'as pas une complexité linéaire en fonction du nombre de ligne des tables. Donc supprimer des lignes le plus tôt possible diminuera la complexité de la jointure. Aussi les jointure on tendances à créés plus de ligne qu'il n'y en avais avant donc la selection a tendance a être plus rapide quand elle est faites avant. Enfin, le multi-threading

permet de faire ces Selection en même temps alors que la selection sur les tuples recrée après la jointure ne le permet pas forcément.

La description rapide d'un algorithme permettant de descendre les sélections ferait un parcours post-fixe sur les noeuds de l'arbre, en vérifiant si à un noeud donnée, elle ne pourrait pas découper l'arbre binaire d'expression au niveau d'un "AND"

2.2 Descente de Projection

Une des difficultés de l'opération de jointure et de sélection est de maintenir les tuples de données valide à chaque opération. En fonction du nombre de colonnes chargées cette opération peut prendre plus ou moins de temps, il est donc intéressant de placer des projections, qui ont un coût très faible devant le coût des jointures et des sélections, le plus tôt possible pour diminuer au plus possible de maintenir ou de travailler sur des données inutiles.

Ainsi, puisque l'ajout de projection après la dernière utilisation d'une colonne ne change pas le fonctionnement d'un plan de requête, on va considérer l'optimisation qui consiste à ajouter la projection la plus stricte au sens des colonnes non-supprimées deux opérations de jointure

Le fonctionnement grossier d'un tel algorithme est de partir de la racine de l'arbre, et de garder en mémoire toutes les colonnes utilisées par le noeud et ses parents. A chaque noeud A, si une colonne non-utilisée par B et ses parents est utilisée par le noeud A, on rajoute une projection enlevant cette colonne entre A et B, puis on continue sur les enfants de A avec la nouvelle liste de colonnes utilisées envoyée aux enfants en même temps.

Les colonnes et les opérations sur celle-ci étant généralement très rapides comparés au nombre de données totales d'une base de données. Les vérifications nécessaires de cet algorithme sont donc très rapides, il est donc lui aussi très rapide.

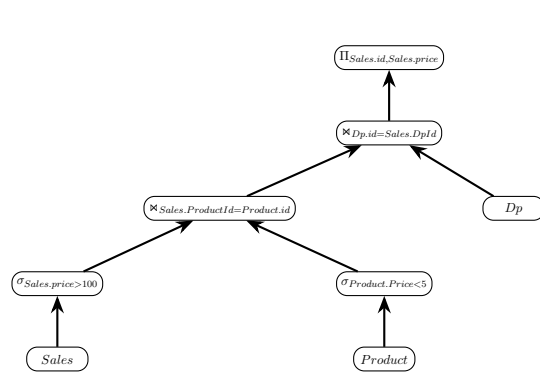


FIGURE 4 – Exemple d’arbre où l’insertion d’une projection après la première jointure pourrait être utile

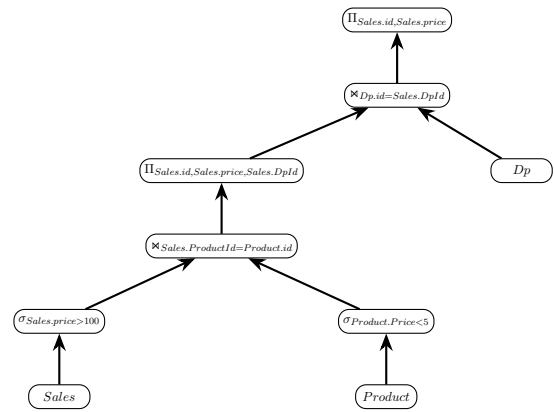


FIGURE 5 – Optimisation de l’arbre en 2.2 en insérant une projection après la première jointure

Dans cet exemple l’insertion d’une projection permettras de gagner du temps sur la futur jointure et le rest de la query et les deux arbres restent équivalent.

3 Optimisation dépendante des données

Il existe des optimisations très intéressante dans certain cas mais néfaste dans d’autre. Les Optimiseur de plan utilisé par les plus grandes entreprises utilisent des modèle pour utilisé des données sur les données stockées pour faire des plans adaptée. Nous montrons les principales optimisations qui peuvent être utilisé par les SGBD pour optimisé la vitesse d’exécution de leurs query

3.1 Optimisation des Sélections

Pour évaluer une condition représenté sous forme d’une expression binaire, on utilise la propriété de paresse, c’est à dire que si en dessous d’un AND on trouve un Faux, on ne vas pas vérifier l’autre côté car le AND seras d’office faux. De même avec un OR si l’on trouve vrai on vas directement renvoyer vrai.

Ainsi, il peut être intéressant d’optimiser l’arbre binaire pour minimiser le nombre d’évaluation. Pour cela il faut tester la conition sur un echantillon de nos données pour estimer à quelle point une condition vas être verifié, puis la ré-arranger pour respecter la condition expliqué au dessus.

Il doit cependant être noté que malgré qu’une comparaison A soit plus facilement verifié qu’une condition B. La vitesse d’évaluation devrait aussi être prise en compte, en effet comparer une string et un booléen sont clairement deux opération qui ne nécessitent pas le même temps d’évaluation et donc les inverser peut

être contre-productive, les modèles les plus récents tiennent donc en facteur cette information aussi lors du choix de l'ordre d'évaluation.

Supposons la requête :

```
1 SELECT S.id, S.price
2 FROM Sales
3 JOIN Dp ON Dp.id = S.DpId
4 Join Product ON S.ProdId = Product.Id
5 WHERE (Dep.pays = "France" AND Sales.Id = 543 ) OR
   Product.Origine = "Chine";
```

Nous obtenons alors en traduction direct de la condition en Expression Binaire.

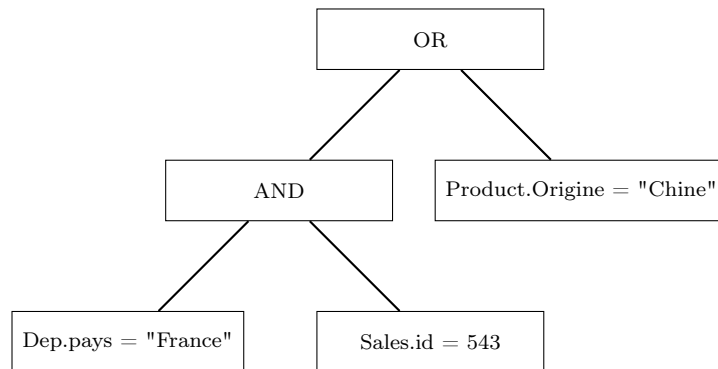


FIGURE 6 – Representation de l'Expression Binaire

Supposons maintenant qu'un très gros nombre de produit sont produit en chine et que par ailleurs il y a un très gros nombre de departement de vente du produit qui sont français mais que le LOT numéro 543 est très peu choisis.

Alors après avoir reconstruit le tuple, on vas d'abord évaluer la condition "Dep.Pays = "France"" ce qui vas être très souvent vrai, alors seulement ensuite on vas vérifier si le numéro de vente est le 543, ce qui vas être très peu souvent vrai alors enfin on vas tester si l'origine du produit est chinoise ce qui vas évidemment très souvent être vrai, alors pour la majorité des produit chinois, on aurais fait deux test inutile. On vas alors réarranger la condition pour obtenir :

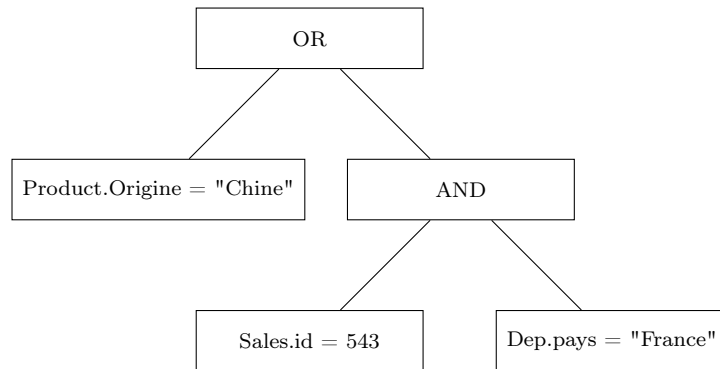


FIGURE 7 – Version optimisé de la condition 3.1

Ici la majorité des tuples vont s’arrêter sur la vérification de l’origine du produit et pour les rares produits non chinois, on va d’abord vérifier l’id de la vente qui est bien plus rarement vérifier que l’origine du département de vente

Le code d’une fonction faisant ceci ferait un parcours post-fixe sur l’entiereté de l’arbre, à noeud, si la condition est vérifiée, on ajoute 1 à son compteur personnel. puis par un parcours pre-fixe, on applique les règles décrites plus haut.

Les Descendances de sélection n’impactant pas la forme générale de l’expression binaire, ces deux opérations sont indépendantes et peuvent radicalement changer la vitesse d’exécution.

3.2 Choix de l’algorithme de jointures

Maintenant que le choix du plan a été fait, il faut l’exécuter. Les optimisations restantes dépendent de la vitesse d’exécution de nos opérations. Une projection est très rapide car elle travaille uniquement sur les colonnes et non sur les données. Les sélections ne possèdent pas une grande variété d’optimisation, le seul cas particulier est celui où nos données sont triées et que la condition se résume à une seule comparaison sur la colonne triée, une recherche dichotomique peut être effectuée.

La partie la plus importante car souvent la plus lente est la jointure, cette opération génère beaucoup de ralentissement, nous présentons ici 4 manières de faire l’opération de jointure, dont trois ont été implémentées avec succès. L’opération de jointure se résume simplement en la création de tout le couple du produit cartésien entre deux tables qui respectent une condition (En SQL cette condition se résume très souvent en une égalité entre deux colonnes).

L’idée naïve pour faire cette opération serait de faire deux boucles imbriquées créant et testant tous les couples possibles et les vérifiant, cette implémentation, appelée Nested Loop est très souvent très lente, sauf dans de rares cas où la création des structures utilisées est plus lente que l’utilisation de boucles imbriquées. Cette implémentation est en $O(n*m)$

Une meilleur idée serait d'effectuer un pré-tri sur les colonnes comparées, pour ensuite faire une lecture en simultan  des deux colonnes et ne garder que les couple verifiant la condition d' galit .

Une id e similaire, non impl ment , serait d' tendre cette logique sur plus de 2 table jointe   la fois. En supposans que l'on fait plusieurs jointure sur la m me colonne de mani re direct ou indirect, on peut alors appliquer cette logique

3.3 Optimisation de l'ordre d'ex cution des jointures

x

Toujours dans une optique de minimiser le nombre de ligne   chaque  tape on peut noter que la jointure est associative c'est   dire avec les tables A, B et C :

$$\bowtie_{C.c = A.a} (\bowtie_{C.d = B.b} (C, B), A) \iff \bowtie_{C.d = B.b} (\bowtie_{C.c = A.a} (C, A), B)$$

Reprenons la requ te de la partie 3.1 et supposons que la table Sales poss de 10^6 entr es, que la table Product poss de 10^4 entr es et enfin que la table D partement poss de 1 entr e. Supposons aussi que la jointure entre Sales et Product produisent 10^6 entr es, la jointure entre Sales et D partement produisent 10^3 entr es et que enfin apr s les deux jointures on obtient 10^2 entr e. Comparons alors les deux mani re d'ex cuter cette double jointure :

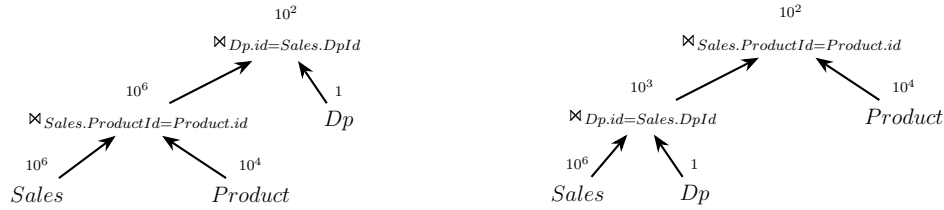


FIGURE 8 – Premi re mani re de faire la double jointure

FIGURE 9 – Deuxieme mani re de faire la double jointure

Supposons que la m thode jointure des deux tables soit un na vement des boucle

imbriqu es qui testent tout les tuples sur la condition, la premi re mani re de faire fait $10^4 \times 10^6 + 10^6 \times 1$ comparaison, tandis que la deuxi me mani re de faire fait $10^6 \times 1 + 10^3 \times 10^4$ comparaisons. La deuxi me mani re de faire minimise donc le nombre total de comparaisons.

Choisir l'ordre des jointures permet donc d'acc l rer la vitesse d'ex cution. Cette m thode permet de dire a posteriori lequel de ces deux ordres de jointure est le plus optimis . La difficult  r side donc dans l'estimation de l'ordre   choisir pour optimiser la vitesse d'ex cution.

Nous avons choisi pour cela d'estim  le rapport entre le nombre de tuple que produirais la jointure et le nombre maximal de lignes entre les deux tables que

nous nommons Rapport de Cardinalité (RC). Nous avons utilisé cette heuristique pour classer les différentes jointures à faire en fonction de leur RC. Nous reconstruisons alors l'arbre en utilisant d'abord les jointures minimisant leur RC. Cette implémentation reste très limitée et sera plus finement analysée dans la partie [4.2](#).

4 Nos Résultats

4.1 Présentation du jeu de donnée

4.2 Analyse des résultats

5 Conclusion

5.1 Conclusion

5.2 Amélioration du modèle actuel

5.3 Autre Champ d'étude pour l'optimisation de notre SGBD

6 Annexe

6.1 Présentation Sommaire du code

6.2 Gestion globale

6.3 Gestion mémoire des tables

6.4 Gestion du plan

6.5 Descriptions rapides du code utilisé